

Artificial Intelligence – SS 2026

Lab Report 1: AI Project Connect Four

Martin Scheuchenpflug, Niklas Etzlinger

May 6, 2026

Abstract

This report documents the experimental analysis of the Minimax algorithm with alpha-beta pruning, applied to the game Connect Four. Five progressively refined versions of the `MinimaxPlayer` are evaluated by measuring the number of board positions examined per move at search depth 4. The modifications include removing pruning entirely, tightening the pruning condition to include equal values, enabling deep (full-window) alpha-beta pruning, and applying a move-ordering heuristic that prefers centre columns.

1.1 Measuring the Original Version

The original `MinimaxPlayer` uses *shallow* alpha-beta pruning: each recursive call passes only the immediate parent's current best value as a bound, discarding information from higher ancestors. The search depth is set to 4 (the application default) throughout all experiments.

The number of positions examined by the computer per move was recorded for 10 consecutive moves of a representative game (Table 1.1). In the original version the counts range from roughly 650 to 2400, depending on how many cutoffs the current board state enables.

1.2 Pure Minimax – No Pruning

1.2.1 Code change

Both alpha-beta conditions were commented out so that no branch is ever cut:

```
// in expandMaxNode
// FIXME: a-b prune
// if (strength > parentMinimum) {
//   board.undoLastMove();
//   return strength;
// }

// in expandMinNode
// FIXME: a-b prune
// if (strength < parentMaximum) {
```

```
// board.undoLastMove();
// return strength;
// }
```

1.2.2 Formula for the exact move count

With pruning disabled the algorithm visits every node in the search tree. The board has $n = 8$ columns. Because each column holds 6 rows, all 8 columns remain available for the first many plies, so the branching factor is a constant 8 throughout the tree.

The root (`calculateMove`) tries 8 moves and calls `expandMinNode`; each of those tries 8 moves and calls `expandMaxNode`; and so on for d levels in total. The total node count is therefore the sum of a geometric series with ratio 8:

$$\text{total}(d) = 8 + 8^2 + 8^3 + \dots + 8^d = \frac{8^{d+1} - 8}{7}. \quad (1.1)$$

For the default depth $d = 4$:

$$\frac{8^5 - 8}{7} = \frac{32768 - 8}{7} = \frac{32760}{7} = 4680.$$

The measured values confirm this: the first three moves of Question 2 all yield exactly 4680 (Table 1.1). Later moves fall slightly below 4680 because some search paths encounter a terminal position (four-in-a-row) before reaching full depth and return without expanding further.

1.3 Pruning with Equal Values

1.3.1 Code change

The original pruning uses strict inequalities: a value *equal* to the parent's bound does not trigger a cutoff. Replacing $>$ with $\geq$ (and $<$ with $\leq$) prunes these tie cases as well:

```
// expandMaxNode: beta cutoff now also fires on equality
if (strength >= parentMinimum) {
    board.undoLastMove();
    return strength;
}

// expandMinNode: alpha cutoff now also fires on equality
if (strength <= parentMaximum) {
    board.undoLastMove();
    return strength;
}
```

1.3.2 Effect

The move counts decrease consistently compared to Question 1 (Table 1.1). Ties are frequent early in the game: the heuristic assigns equal scores to many symmetric positions, so the additional cutoffs on equal values can eliminate entire subtrees that the strict version would have explored.

1.4 Deep Alpha-Beta Pruning

1.4.1 Motivation

In the original code each function passes only one bound to its child – either the parent MAX node’s current best (alpha) or the parent MIN node’s current minimum (beta). Information from grandparents and higher ancestors is silently discarded. *Deep* alpha-beta threads both bounds through the entire tree: every node sees the tightest window established by *all* its ancestors.

1.4.2 Code change

Both recursive functions were given a full (α, β) signature and update their respective bound locally before passing both values down:

```
private int expandMaxNode(int depth, int alpha, int beta) {
    ...
    int strength = expandMinNode(depth - 1, alpha, beta);
    if (strength >= beta) {           // beta cutoff
        board.undoLastMove();
        return strength;
    }
    if (strength > maxStrength) maxStrength = strength;
    if (strength > alpha) alpha = strength; // tighten window
    ...
}

private int expandMinNode(int depth, int alpha, int beta) {
    ...
    int strength = expandMaxNode(depth - 1, alpha, beta);
    if (strength <= alpha) {         // alpha cutoff
        board.undoLastMove();
        return strength;
    }
    if (strength < minStrength) minStrength = strength;
    if (strength < beta) beta = strength; // tighten window
    ...
}
```

The root call initialises the window to the widest possible range:

```
int strength = expandMinNode(depth - 1, maxStrength, MAX_POSSIBLE_STRENGTH);
```

1.4.3 Effect

Move counts drop further compared to Question 3 (Table 1.1). Whenever the (α, β) window is tightened at any level, all descendants benefit immediately rather than only the direct children. The gain is most visible in positions where good moves appear early and establish tight bounds quickly.

1.5 Move-Ordering Heuristic

Alpha-beta pruning is most effective when the best moves are considered first, because early good moves narrow the (α, β) window quickly and cause more cutoffs later. For Connect Four, centre columns are generally stronger than edge columns. A fixed middle-out order was implemented:

```
// 8-column board: centre pair first , alternating outward
private static final int[] COLUMN_ORDER = {3, 4, 2, 5, 1, 6, 0, 7};

private Move[] reorderMoves(Move[] moves) {
    Move[] ordered = new Move[moves.length];
    for (int i = 0; i < COLUMN_ORDER.length; i++) {
        ordered[i] = moves[COLUMN_ORDER[i]];
    }
    return ordered;
}
```

`reorderMoves` is called at all three loop sites: `calculateMove`, `expandMaxNode`, and `expandMinNode`.

1.5.1 5a – Pure Minimax with Middle-Out Ordering

With alpha-beta pruning disabled the traversal order has no effect on the *number* of nodes visited: every node is expanded regardless. The move counts therefore remain close to 4680 and follow the same formula as Section 1.2 (Table 1.1). Minor deviations from 4680 appear in later moves because the computer now plays different moves (equal heuristic values are resolved in a different order), leading to a different board state and different numbers of full columns.

1.5.2 5b – Deep Alpha-Beta with Middle-Out Ordering

Combining deep alpha-beta (Question 4) with middle-out ordering produces the lowest move counts of all variants (Table 1.1). By trying the best columns first, the algorithm establishes a tight (α, β) window early and cuts off the remaining columns with minimal effort. The reduction is dramatic – move 1 drops from 993 (Question 4) to just 302.

Results Summary

Table 1.1 collects all measurements. Notes on comparability: Questions 1–4 were measured on the same sequence of human moves (the computer responds identically in all versions that use the same evaluation order). Questions 5a and 5b diverge from move 2 onwards because the computer’s choices change under middle-out ordering.

Table 1.1: Number of moves examined per computer turn at depth 4. Columns correspond to the six experimental variants.

Move	Q1 original	Q2 pure minimax	Q3 $\geq$ prune	Q4 deep a-b	Q5a pure + order	Q5b deep + order
1	1426	4680	1208	993	4680	302
2	925	4680	771	642	4680	360
3	654	4680	574	514	4650	702
4	809	4623	745	612	4538	899
5	1221	4622	1158	934	4650	872
6	2459	4056	2007	1936	4649	773
7	909	4544	854	754	4333	785
8	760	4256	736	664	4325	671
9	899	3978	793	665	2792	305
10	664	3641	59	59	2347	95

1.6 Reducing to 7 Columns

1.6.1 Changing the column count

Standard Connect Four is played on a 6×7 board. The original code uses `NUMBER_OF_COLUMNS = 8`. Changing this constant to 7 is sufficient to play the game correctly; all array sizes and loop bounds throughout `C4Board.java` are derived from the constant at compile time.

```
public static final int NUMBER_OF_COLUMNS = 7; // was 8
```

1.6.2 The magic number 84

One line in `C4Board.java` is not derived from the constants:

```
rows = new Vector<C4Row>(84);
```

This sets the initial capacity of the vector that stores every possible group of four consecutive slots – the groups used by the heuristic to evaluate the board. The number 84 is the exact count of such groups on a 6×8 board, computed direction by direction:

Direction	Formula	6×8
Horizontal	$R \cdot (C - 3)$	$6 \cdot 5 = 30$
Vertical	$(R - 3) \cdot C$	$3 \cdot 8 = 24$
Diagonal ↗	$(R - 3) \cdot (C - 3)$	$3 \cdot 5 = 15$
Diagonal ↘	$(R - 3) \cdot (C - 3)$	$3 \cdot 5 = 15$
Total		84

The general formula in terms of `NUMBER_OF_ROWS` (R) and `NUMBER_OF_COLUMNS` (C) is:

$$R(C - 3) + (R - 3)C + 2(R - 3)(C - 3). \quad (1.2)$$

For the standard 6×7 board this gives $6 \cdot 4 + 3 \cdot 7 + 2 \cdot 3 \cdot 4 = 24 + 21 + 24 = 69$.

The fix replaces the hard-coded literal with the formula so it remains correct for any board size:

```
rows = new Vector<C4Row>(
    NUMBER_OF_ROWS * (NUMBER_OF_COLUMNS - 3)
    + (NUMBER_OF_ROWS - 3) * NUMBER_OF_COLUMNS
    + 2 * (NUMBER_OF_ROWS - 3) * (NUMBER_OF_COLUMNS - 3));
```

The move-ordering array in `MinimaxPlayer.java` was also updated from the 8-element sequence to the correct 7-element centre-out order:

```
// 7-column board: centre column 3 first, alternating outward
private static final int[] COLUMN_ORDER = {3, 2, 4, 1, 5, 0, 6};
```

After these three changes the game runs correctly on the standard 6×7 board.

1.7 The Heuristic Evaluation Function

1.7.1 What quantities does it compute, and how are they weighted?

The heuristic counts *open groups*: groups of 4 consecutive slots (horizontal, vertical, or diagonal) that contain tokens of only one player—the other player has placed no token in the group yet, so it can still be completed into a four-in-a-row. A group with tokens of both players is *dead* and counted for neither. For each player the number of open groups of size 1, 2, and 3 is tracked separately.

The raw strength of a player is computed in `C4Stats`:

```
// SHIFT_3_IN_A_ROW = 5 => x32
// SHIFT_2_IN_A_ROW = 2 => x4
return (p1_2InARow << SHIFT_2_IN_A_ROW)
      + (p1_3InARow << SHIFT_3_IN_A_ROW)
      + p1_1InARow;
// i.e. 32*(3-in-a-row) + 4*(2-in-a-row) + 1*(1-in-a-row)
```

The comment in the source claiming multipliers of 64 and 16 is outdated; the actual weights are **32:4:1**. Two special cases override the formula: a player with 4-in-a-row yields `Integer.MAX_VALUE` (win); if the *opponent* has 4-in-a-row the raw strength is 0. The value returned to minimax is always the difference:

$$\text{getStrength}(p) = \text{strength}_p - \text{strength}_{\bar{p}}.$$

1.7.2 How is it computed? Object communication

The evaluation is never recomputed from scratch. It is maintained *incrementally* through a chain of event notifications that follow the Observer pattern.

C4Slot is the *subject*. It stores the content of one cell and holds a list of `C4SlotListener` objects. Whenever `setContents()` is called it fires `contentsChanged(old, new)` on every registered listener.

C4Row is the *observer*. Each instance represents one group of 4 consecutive slots and implements `C4SlotListener`. Its constructor registers it on all four slots. It maintains counters `numPlayer1` and `numPlayer2`. On notification it updates these counters and calls the matching `Inc/Dec` method on the shared `C4Stats` object—for example, if player 1 now has 2 tokens in the group and the opponent has none, it calls `stats.p1_1InARowDec()` followed by `stats.p1_2InARowInc()`.

C4Stats is the *global accumulator*. It holds board-wide counters that `getStrength()` simply reads—evaluation costs $O(1)$. A slot can participate in up to 11 overlapping groups, which is why `C4Slot` pre-allocates its listener array with capacity 11.

The full call chain on every `board.move()` is:

```
C4Board.move()
-> slot.setContents(playerNumber)
-> notifies each listening C4Row
-> row.contentsChanged() updates numPlayer1 / numPlayer2
-> calls stats.pX_NInARowInc() or Dec()
```

`undoLastMove()` triggers the same chain in reverse, keeping all counters consistent without any board scan.

1.7.3 The restart bug

The counter fields in `C4Stats` are declared `static`, making them class-level variables shared across *all* instances. When `C4Board.clone()` creates `clone.stats = new C4Stats()`, the new object still reads and writes the same static counters as the original board. The board-state copy loop then calls `setContents()` on the clone's slots, which fires the clone's freshly created `C4Row` listeners and increments the shared counters a second time. The current board state is thus added as a constant offset to every counter.

Because the offset is identical for every leaf node in a single minimax call, the relative ordering of moves is unaffected—the extra constant cancels in the difference $\text{strength}_p - \text{strength}_{\bar{p}}$. The bug is therefore harmless in practice.

1.8 Missing the Immediate Win

Sometimes the computer can win in one move but plays a different move instead. The root cause is that the heuristic assigns *exactly the same score* to every winning position regardless of how many moves away the win is: `firstPlayerStrength()` returns `Integer.MAX_VALUE` whenever `p1_4InARow != 0`, whether the win occurred at depth 1 or depth 3.

The move-selection loop in `calculateMove` updates its best move only on a *strict* improvement:

```
if (strength > maxStrength) {
    maxStrength = strength;
    maxIndex = i;
}
```

Consider the following situation at depth 4 with left-to-right move ordering:

1. Move at column 0 is tried first. The search finds a forced win in 3 more moves somewhere in the tree and returns `Integer.MAX_VALUE`. `maxIndex = 0`.
2. Move at column 3 is the immediate one-move win. After placing the piece `board.isGameOver()` is true, so `expandMinNode` returns `Integer.MAX_VALUE` immediately.
3. `Integer.MAX_VALUE > Integer.MAX_VALUE` is **false**, so `maxIndex` remains 0.
4. The computer plays column 0—a winning move, but not the immediate one.

The computer still wins the game; it simply does not do so as quickly as possible. A fix would be to encode the win depth in the score, e.g. `Integer.MAX_VALUE - depth`, so shallower wins always outrank deeper ones. The current code makes no such distinction.

Summary and Comments

The experiments clearly show the cumulative effect of each optimisation. Removing pruning (Q2) inflates the count to the theoretical maximum of $\frac{8^5-8}{7} = 4680$ nodes. Restoring shallow pruning with equal-value cutoffs (Q3) already reduces counts substantially below the original (Q1). Deep alpha-beta (Q4) cuts further by propagating bounds from all ancestors. Finally, middle-out move ordering (Q5b) delivers the largest single reduction: starting from the best columns narrows the search window immediately and causes the remaining columns to be pruned with far less work.

The Q5a result confirms the theoretical expectation that move ordering is irrelevant for pure minimax – the counts stay near 4680 regardless of column order, with small deviations caused by the diverging game state.

Question 6 shows that a single constant controls the board width throughout the program, with the exception of the hard-coded vector capacity 84. That literal encodes the number of four-in-a-row groups on the original 6×8 board; its general form (Equation 1.2) evaluates to 69 for the standard 6×7 configuration.

Question 7 reveals an elegant incremental design: the heuristic is maintained at $O(1)$ cost per move through the Observer pattern, with `C4Row` objects listening to individual slots and propagating changes into a shared `C4Stats` accumulator.